

Appunti di Ingegneria del Software

Appunti basati sulle lezioni di Davide Falessi

A.A. 2025/2026

Indice

1	Teoria della Misurazione e Scale di Misura	4
1.1	Introduzione alla Misurazione del Software	4
1.1.1	Definizioni Formali	4
1.1.2	Proprietà di una Metrica di Qualità	4
1.2	Tassonomia delle Scale di Misura	5
1.2.1	Scala Nominale e Ordinale	5
1.2.2	Scale Numeriche (Intervallo e Rapporto)	5
1.3	Modelli di Qualità: Lo Standard ISO/IEC 25010	6
1.4	Introduzione alle Metriche Object-Oriented	6
2	GQM+S, Stima dei Costi e Planning Poker	7
2.1	GQM+Strategies (Goal-Question-Metric + Strategies)	7
2.1.1	Il Reference Process (I 6 Step del GQM+S)	7
2.2	Introduzione alla Stima dei Costi	8
2.2.1	Componenti del Costo ed Overheads	8
2.2.2	Il Ruolo del Linguaggio di Programmazione	9
2.3	Tecniche di Stima	9
2.4	Planning Poker	9
2.4.1	Dinamiche e Regole del Gioco	10
2.4.2	Vantaggi e Consigli Pratici (Tips)	10
3	Introduzione all'Architettura Software	11
3.1	Cos'è un'Architettura Software	11
3.1.1	Architetture Intended vs Unintended	11
3.2	Il Modello 4+1 di Kruchten	12
3.3	Progettazione e Decision-Making	12
3.3.1	L'importanza dei Requisiti Non Funzionali	12
3.3.2	Trade-offs e il ruolo dell'Architetto	13
3.4	Documentazione: Viste e Punti di Vista	13
3.5	Software Product Lines (SPL)	13
3.6	Stili Architettureali	14
4	Debito Tecnico e Code Smells	16
4.1	Il Debito Tecnico (Technical Debt)	16
4.1.1	Cause e Conseguenze	16
4.1.2	Strategie di Gestione	17
4.2	Introduzione ai Code Smells	17
4.3	Catalogo dei "Worst Smells"	17
4.3.1	Esempi di Worst Smells	18

5	Machine Learning e Software Analytics per la Software Engineering	19
5.1	Definizione di Machine Learning, ruolo di statistica e informatica	19
5.2	Esempi d'uso: associazioni, classificazione, regressione	19
5.2.1	Apprendimento di associazioni	19
5.2.2	Classificazione	20
5.2.3	Regressione	20
5.3	Tipi di apprendimento: supervised, unsupervised, semi-supervised, reinforcement	20
5.3.1	Supervised learning	20
5.3.2	Unsupervised learning	21
5.3.3	Semi-supervised learning	21
5.3.4	Reinforcement learning	21
5.4	Etica del data mining e del ML	21
5.5	Terminologia: istanze, feature, attributi nominali/numerici, output, modelli	22
5.5.1	Instances	22
5.5.2	Features o attributi	22
5.5.3	Output e modelli	22
5.6	Software quality e costo dei bug	23
5.7	Software analytics: definizione, obiettivi e fonti dati	23
5.8	AI/ML applicati alla Software Engineering	24
5.8.1	Applicazioni alla qualità	24
5.8.2	Applicazioni alla produttività	24
5.9	Defect prediction e ML for software defects	25
5.10	Pipeline: measure, clean data, analyze, model, explain	25
5.11	Estrazione dati da ITS e VCS e raccolta metriche	26
5.12	Identificazione difetti, labeling e defect dataset	26
5.13	Black-box models e necessità di spiegazione	27
5.14	XAI, GDPR, fairness, accountability, transparency	27
5.15	CMMI Level 5 e process improvement	28
5.16	Process chart e controllo statistico	28
5.17	Casi applicativi mostrati nelle slide, anche se solo introdotti	29
5.17.1	Simulatore di release e defect prediction	29
5.17.2	Traceability recovery	29
5.17.3	Defectiveness prediction via JIT	30
5.18	Riepilogo finale	30
6	SZZ: Identifying Buggy Entities	31
6.1	Contesto	31
6.1.1	Cos'è la defect prediction	31
6.1.2	Perché la qualità del dataset è fondamentale	31
6.1.3	Principali fonti di rumore nei dataset	31
6.2	Cos'è SZZ	32
6.2.1	Definizione e obiettivo	32
6.2.2	Uso del versioning e dell'annotation	32
6.2.3	Idea di partire dal fix per risalire al bug-introducing change	32
6.3	Come funziona SZZ	33
6.3.1	Identificazione del fix commit	33
6.3.2	Analisi delle linee modificate	33

6.3.3	Ricerca all'indietro dell'ultima modifica precedente	33
6.3.4	Differenza tra bug-fix change e bug-introducing change	33
6.4	Esempi nelle slide	34
6.4.1	Esempio 1: "When Do Changes Induce Fixes?"	34
6.4.2	Esempio 2: "The Impact of Refactoring Changes on the SZZ Algorithm: An Empirical Study"	34
6.5	Limiti di SZZ	34
6.5.1	Code additions	34
6.5.2	Regressive bugs	35
6.5.3	Refactoring	35
6.5.4	Imprecise backward search	35
6.5.5	Snoring	35
6.6	Da ricordare per l'esame	35
7	Leveraging Defects Lifecycle for Labeling Defective Classes	37
7.1	Contesto e obiettivo del lavoro	37
7.1.1	Defect prediction	37
7.1.2	Importanza del dataset	37
7.1.3	Labeling delle classi difettose	37
7.1.4	Obiettivo generale dello studio	37
7.2	Concetti fondamentali	38
7.2.1	SZZ	38
7.2.2	Earliest AV in JIRA	38
7.2.3	Defect lifecycle	38
7.2.4	Significato di IV, OV, FV e AV	38
7.3	SZZ: funzionamento e limiti	38
7.3.1	Come funziona SZZ	38
7.3.2	Uso di <code>git blame</code> / annotation mechanism	39
7.3.3	Bug introducing commit	39
7.3.4	Limiti del metodo	39
7.4	Aim dello studio e Research Questions	39
7.4.1	Idea di stable life-cycle dei defect	39
7.4.2	Research Questions	39
7.5	RQ1: Is the AV available and consistent?	40
7.5.1	Razionale	40
7.5.2	Design e selezione di progetti e bug	40
7.5.3	Risultati	40
7.5.4	Discussione e implicazioni pratiche	40
7.6	RQ2: Do methods have different accuracy in AV labeling?	41
7.6.1	Razionale	41
7.6.2	Formula della Proportion e significato di P	41
7.6.3	Varianti di Proportion	41
7.6.4	Metodi confrontati	42
7.6.5	Metriche	42
7.6.6	Risultati e discussione	42
7.7	RQ3: Do methods have different accuracy in classes labeling?	43
7.7.1	Differenza rispetto a RQ2	43
7.7.2	Design	43

7.7.3	Risultati	43
7.7.4	Discussione	43
7.8	RQ4: Do methods lead to different identification of important features? . .	44
7.8.1	Razionale	44
7.8.2	Feature selection	44
7.8.3	Correlation analysis e Standard Mean Relative Error	44
7.8.4	Risultati	44
7.8.5	Discussione	44
7.9	Figure, tabelle e workflow del paper	44
7.9.1	Diagramma del defect lifecycle	45
7.9.2	Workflow del predicted AV	45
7.9.3	Workflow del class labeling	45
7.9.4	Workflow dei complete datasets	45
7.9.5	Feature selection e correlation analysis	45
7.9.6	Interpretazione di grafici, boxplot e tabelle	45
7.10	Conclusioni e future work	46
7.10.1	Risultati principali del paper	46
7.10.2	Perché AV non basta	46
7.10.3	Perché Proportion è rilevante	46
7.10.4	Significatività statistica vs rilevanza pratica	46
7.10.5	Sviluppi futuri	46

Capitolo 1

Teoria della Misurazione e Scale di Misura

1.1 Introduzione alla Misurazione del Software

La misurazione nel software engineering non è un fine, ma uno strumento decisionale di supporto. Come ricordano Lord Kelvin e Tom DeMarco, l'assenza di misurazione impedisce il controllo e, di conseguenza, il miglioramento dei processi e dei prodotti[cite: 330, 341].

1.1.1 Definizioni Formali

Secondo i modelli standard, è necessario distinguere tre concetti fondamentali:

- **Misura (Measure):** Rappresentazione numerica di una caratteristica o di un attributo di un prodotto o processo software (es. 5000 LOC, 30 mesi-uomo)[cite: 363, 364, 365, 368].
- **Metrica (Metric):** La scala o il sistema di misurazione utilizzato per quantificare una misura[cite: 352]. Definisce le regole di assegnazione dei valori (es. Complessità Ciclomatica, Densità dei Difetti)[cite: 353, 354].
- **Misurazione (Measurement):** Il processo pratico di assegnazione dei numeri alle misure utilizzando una metrica[cite: 357].

1.1.2 Proprietà di una Metrica di Qualità

Per essere considerata valida, una metrica deve soddisfare i seguenti criteri:

Rilevanza: Deve misurare un attributo significativo e importante del prodotto o processo software[cite: 380].

Oggettività: Il risultato deve basarsi su dati verificabili e riproducibili, non su opinioni o stime soggettive[cite: 381, 382].

Affidabilità e Consistenza: La metrica deve produrre risultati costanti in condizioni diverse e deve essere coerente con gli obiettivi del progetto[cite: 383, 384].

Usabilità: Il calcolo e l'interpretazione devono essere facili, intuitivi e non richiedere sforzi o risorse eccessive[cite: 385, 386].

Nota del Prof

La pertinenza di una metrica è strettamente legata al contesto di business. Un errore critico in un sistema assicurativo potrebbe riguardare l'accuratezza dei calcoli (oracolo) con conseguenze disastrose, mentre per un motore di ricerca come Google la priorità assoluta risiede nella disponibilità del servizio (uptime), dove anche poche ore di downtime globale risultano inaccettabili.

1.2 Tassonomia delle Scale di Misura

La scelta della scala utilizzata dipende dalla natura dei dati misurati e determina quali operazioni statistiche siano matematicamente applicabili[cite: 407, 459, 477].

1.2.1 Scala Nominale e Ordinale

- **Nominale:** Utilizzata per assegnare valori a etichette o categorie senza un ordine intrinseco o magnitudine (es. tipi di difetti: logica, UI, performance, sicurezza)[cite: 417, 418, 419]. Operazioni ammesse: Moda, frequenza[cite: 459].
- **Ordinale:** Prevede un ordine di rango per le categorie, ma la distanza o magnitudine tra i valori non è definita né significativa (es. priorità dei difetti: Critical, High, Medium, Low)[cite: 422, 423, 424]. Operazioni ammesse: Mediana[cite: 459].

Nota del Prof

Le scale ordinali, pur limitando le analisi statistiche alla mediana, offrono spesso una maggiore oggettività rispetto a scale numeriche forzate. Valutare la severità di un bug su una scala 0-100 introduce un'alta soggettività (es. la differenza tra 65 e 70 è arbitraria), mentre una distinzione qualitativa (Blocker vs Non-Blocker) risulta più robusta, netta e oggettiva.

1.2.2 Scale Numeriche (Intervallo e Rapporto)

In queste scale i valori sono assegnati in ordine di rango e la magnitudine della differenza tra i punti ha un significato reale e misurabile[cite: 427, 433].

- **Intervallo:** La magnitudine delle differenze è significativa, ma non possiede uno zero assoluto reale[cite: 427, 428].
- **Rapporto (Ratio):** È il tipo di scala più sofisticato, in cui la magnitudine delle differenze è calcolabile e possiede uno zero assoluto reale (es. LOC, sforzo, tempo)[cite: 433, 434, 435].

Statistiche ammesse per entrambe: Media, Deviazione Standard, Varianza, Range (per la scala di Rapporto sono ammessi anche coefficiente di variazione e media geometrica)[cite: 459].

1.3 Modelli di Qualità: Lo Standard ISO/IEC 25010

Lo standard ISO 25010 definisce una tassonomia dettagliata degli attributi di qualità (qualità del prodotto e qualità in uso)[cite: 33]. Le caratteristiche principali individuate dallo standard sono:

1. **Functional Suitability (Adeguatezza funzionale):** Completezza, correttezza e adeguatezza funzionale[cite: 39, 40, 41, 42].
2. **Performance Efficiency (Efficienza delle prestazioni):** Comportamento temporale, utilizzo delle risorse e capacità strutturale[cite: 43, 44, 45, 46].
3. **Security (Sicurezza):** Riservatezza, integrità, non ripudio, responsabilità e autenticità[cite: 47, 48, 49, 50, 51, 52].
4. **Maintainability (Manutenibilità):** Modularità, riusabilità, analizzabilità, modificabilità e testabilità[cite: 53, 54, 55, 56, 57, 58].

Nota del Prof

È cruciale distinguere sempre il modello di qualità dalle metriche: il modello (come l'ISO 25010) definisce *cosa* significa la qualità fornendone una tassonomia, mentre le metriche definiscono *come* quantificarla operativamente[cite: 30]. L'adozione di questi standard fallisce sistematicamente se manca la "buona fede" aziendale; l'uso delle metriche al solo scopo di ottenere certificazioni formali di facciata, senza mirare a un miglioramento sostanziale del prodotto, svislisce l'utilità dell'intero sistema di valutazione ingegneristico.

1.4 Introduzione alle Metriche Object-Oriented

Le metriche object-oriented vengono impiegate per misurare la qualità dei sistemi software ad oggetti e identificare tempestivamente difetti di progettazione o futuri problemi di manutenibilità[cite: 68, 69]. Vanno sempre interpretate come indicatori di rischio e valutate in modo comparativo (rispetto a una baseline o a un trend) e non in senso assoluto[cite: 73].

Nota del Prof

In ambito manageriale, è molto più efficace adottare un approccio pratico basato sulle distribuzioni piuttosto che sulle medie globali. Risulta strategicamente più utile concentrare le attività di analisi, refactoring e test mirati esclusivamente sul 10% delle classi che presentano i valori peggiori di complessità (*outliers*), piuttosto che tentare di ottimizzare e riscrivere l'intero sistema in modo uniforme.

Capitolo 2

GQM+S, Stima dei Costi e Planning Poker

2.1 GQM+Strategies (Goal-Question-Metric + Strategies)

L'approccio GQM+Strategies (GQM+S) è una metodologia che combina il metodo classico GQM con l'identificazione delle migliori strategie per raggiungere gli obiettivi. Questo approccio collega a cascata gli obiettivi aziendali alla misurazione:

Business Goals → Software Goals → Questions → Measures → Strategies / Targets.

Nota del Prof

Il CQM originale è stato co-creato dal Professor Cantone. L'estensione "+ Strategies" serve ad associare l'azione tecnica al risultato voluto. Tutto parte dal Business: ogni feature o refactoring viene fatto in primis per raggiungere obiettivi aziendali.

2.1.1 Il Reference Process (I 6 Step del GQM+S)

Il processo si sviluppa in sei step principali per permettere il miglioramento continuo:

0. **Initialize:** Impegno (Commitment), Staffing e Responsabilità.
1. **Characterize:** Definire lo scope dell'applicazione e caratterizzare il contesto ambientale.
2. **Set Goals:** Analisi dello status quo, prioritizzazione degli obiettivi e costruzione della griglia goals-strategies.
3. **Choose Process:** Pianificare l'implementazione delle strategie e organizzare la raccolta/analisi dei dati.
4. **Execute Model:** Esecuzione delle strategie, raccolta dati e feedback.
5. **Analyze Results:** Analisi dei dati, revisione delle strategie e analisi costi/benefici.
6. **Package and Improve:** Adattamento dei goal e registrazione delle "lessons learned".

Nota del Prof

L'ultimo step di "packaging" si basa sul concetto di *Experience Factory*: lo scopo è impacchettare l'esperienza in una knowledge base per riutilizzarla in futuro.

Solo nelle slide (non spiegato a voce)

Esempi di utilizzo tipici:

- **Miglioramento del processo:** Ridurre lead time o rework (Metriche: cycle time, difetti sfuggiti).
- **Project management:** Migliorare prevedibilità (Metriche: milestone burndown, throughput).
- **Supporto decisionale:** Valutare l'adozione di nuove tecnologie (Metriche: esiti progetto pilota).

2.2 Introduzione alla Stima dei Costi

La stima si basa su tre domande fondamentali: quanto sforzo (*effort*) è richiesto, quanto tempo di calendario è necessario e qual è il costo totale dell'attività.

Nota del Prof

Bisogna distinguere l'effort (lavoro puramente impiegato) dal tempo di calendario: un'attività può richiedere poco sforzo ma molto tempo se le risorse non sono subito disponibili.

2.2.1 Componenti del Costo ed Overheads

Il costo include Hardware/Software, viaggi, formazione e l'Effort, che è il fattore dominante (salari e costi sociali). Gli **Overheads** includono:

- Costi degli edifici (riscaldamento, luce).
- Costi di rete e comunicazione.
- Strutture condivise come mensa o biblioteca.

Nota del Prof

Il salario dello sviluppatore a volte non copre neanche il 50% del costo totale per un'azienda a causa degli overhead. Inoltre, il software ha un costo marginale di replicazione quasi nullo: vendere a un secondo cliente non costa nulla di più in termini di produzione.

2.2.2 Il Ruolo del Linguaggio di Programmazione

Solo nelle slide (non spiegato a voce)

Il linguaggio impatta drasticamente su tempi e costi. Ad esempio, l'Assembly richiede molto più tempo per coding (8 settimane per 5000 linee) e testing (10 settimane) rispetto a un linguaggio ad alto livello per lo stesso sistema.

2.3 Tecniche di Stima

IMPORTANTE PER L'ESAME

All'esame non vengono chieste le tecniche a memoria, ma i vantaggi, gli svantaggi e quando usare l'una rispetto all'altra.

- **Algorithmic cost modelling:** Approccio basato su formule e dati storici, generalmente basato sulla dimensione del software (es. COCOMO).
- **Expert judgement:** Uno o più esperti usano la loro esperienza per predire i costi.
 - *Vantaggi:* Metodo relativamente economico; accurato se gli esperti hanno esperienza diretta in sistemi simili.
 - *Svantaggi:* Molto inaccurato se mancano veri esperti nel dominio.
- **Estimation by analogy:** Il costo è calcolato comparando il progetto con uno simile nello stesso dominio.
 - *Vantaggi:* Accurato se sono disponibili dati di progetti passati.
 - *Svantaggi:* Impossibile se non si è mai affrontato un progetto comparabile; richiede un database di costi sistematicamente aggiornato.
- **Parkinson's Law:** Il progetto costa qualunque risorsa sia disponibile. Il lavoro si espande per riempire il tempo disponibile.
 - *Vantaggi:* Non si va mai oltre il budget (*no overspend*).
 - *Svantaggi:* Il sistema rimane solitamente incompleto (*unfinished*).
- **Pricing to win:** Il costo è pari a quanto il cliente può spendere.
 - *Vantaggi:* Permette di vincere il contratto.
 - *Svantaggi:* Bassa probabilità che il cliente ottenga ciò che vuole; i costi non riflettono il lavoro reale richiesto.

2.4 Planning Poker

Il Planning Poker è una tecnica Agile per la stima del consenso, basata sulle carte di James Grenning.

2.4.1 Dinamiche e Regole del Gioco

Ogni partecipante riceve un mazzo di carte con una sequenza numerica, solitamente di **Fibonacci** (1, 2, 3, 5, 8, 13, 21, ecc.).

- **La Sfida:** Il moderatore presenta una *user story* e il team può fare domande al Product Owner.
- **Voto Segreto:** Ogni membro seleziona privatamente una carta che rappresenta la "grandezza" (*size*) del task, considerando tempo, rischio e complessità.
- **Svelamento e Dibattito:** Le carte vengono mostrate insieme. Se non c'è consenso, chi ha dato i valori estremi spiega le proprie ragioni. Il gruppo discute e si vota di nuovo finché non si raggiunge un accordo.

IMPORTANTE PER L'ESAME

Perché i "buchi" di Fibonacci? Servono a forzare la granularità corretta. All'aumentare della grandezza aumenta l'incertezza: la sequenza ci spinge a scegliere tra valori distanti (es. 13 o 21), riflettendo la reale imprecisione della stima per task grandi.

Nota del Prof

In caso di mancato consenso, si sceglie la stima superiore per tutelarsi dalla naturale indole ottimista dell'uomo, che tende a sottostimare i tempi ignorando gli imprevisti.

2.4.2 Vantaggi e Consigli Pratici (Tips)

Il Planning Poker favorisce la collaborazione e fa emergere i problemi tecnici in anticipo attraverso il dibattito.

- **Chi vota:** Solo chi farà materialmente il lavoro. I manager non votano per non influenzare le stime al ribasso.
- **Pareggi:** Se il team è diviso tra due taglie consecutive (es. 5 e 8), si sceglie la maggiore e si procede.

Solo nelle slide (non spiegato a voce)

Suggerimenti per l'efficienza:

- Usare un timer per limitare le discussioni a circa un minuto.
- Includere una carta "I need a break" per segnalare la stanchezza.
- Mantenere una **Baseline** consistente tra le varie iterazioni.

Capitolo 3

Introduzione all'Architettura Software

3.1 Cos'è un'Architettura Software

IMPORTANTE PER L'ESAME

L'architettura software è uno dei pilastri dell'ingegneria del software e rappresenta una delle domande più frequenti in sede d'esame.

Secondo lo standard **IEEE 1471**, l'architettura è definita come:

“L'organizzazione fondamentale di un sistema, incarnata nei suoi componenti, nelle loro relazioni reciproche e con l'ambiente, e i principi che ne governano la progettazione e l'evoluzione.”

L'intento principale, come sostenuto da Kruchten, è fornire un **controllo intellettuale** su sistemi di enorme complessità, permettendo agli stakeholder di comprendere la struttura senza annegare nei dettagli implementativi.

3.1.1 Architetture Intended vs Unintended

Ogni sistema possiede intrinsecamente un'architettura, che può essere classificata in due modi:

- **Intended (Pianificata):** Esplicita e documentata prima della scrittura del codice. Segue l'approccio del *Forward Engineering*.
- **Unintended (Accidentale):** Implicita e spontanea. Emerge quando si scrive codice senza un piano; in questo caso, l'architettura deve essere ricostruita a posteriori (*Reverse Engineering*).

Nota del Prof

Immaginate di costruire una casa: nell'approccio *intended* disegnate prima la pianta. Nell'approccio *unintended* costruite le stanze a casaccio e solo alla fine vi accorgete di che forma ha la casa. Ricordate però il Manifesto Agile: l'architettura ha valore solo se serve a produrre codice migliore, non deve essere burocrazia inutile.

3.2 Il Modello 4+1 di Kruchten

Per gestire la complessità, Kruchten propone di decomporre l'architettura in diverse **viste** basate sul principio della *separation of concerns*.

- **Logical View:** Supporta i requisiti funzionali, ovvero ciò che il sistema deve fare per l'utente finale.
- **Development View:** Si concentra sull'organizzazione dei moduli e dei package nel codice sorgente.

Nota del Prof

Un esempio tipico è lo **stack di Android**, che mostra come il sistema sia stratificato dal Kernel Linux fino alle applicazioni.

- **Dynamic View:** Analizza il comportamento a runtime, i processi e le interazioni tra oggetti. Viene solitamente modellata con i *Sequence Diagram*.

Solo nelle slide (non spiegato a voce)

Completamento del modello:

- **Deployment View:** Si occupa della mappatura fisica del software sui nodi hardware e della distribuzione geografica dei server.
- **Scenarios (il “+1”):** Casi d'uso o sequenze di azioni che fanno da collante tra le altre quattro viste, verificandone la coerenza.

Nota del Prof

Pensate a una casa: potete guardare la pianta elettrica, quella idraulica o quella strutturale. Sono tutte viste diverse dello stesso edificio, necessarie a stakeholder diversi (elettricista, idraulico, ingegnere).

3.3 Progettazione e Decision-Making

Il design architeturale si basa su tre fattori: **Riuso** (di pattern o componenti), **Metodo** (tecniche sistematiche) e **Intuizione** (l'esperienza dell'architetto).

3.3.1 L'importanza dei Requisiti Non Funzionali

Nota del Prof

L'architettura è dettata principalmente dai **Requisiti Non Funzionali** (attributi di qualità come sicurezza, velocità, scalabilità). Questi requisiti sono strutturali: non si può “aggiungere la sicurezza” alla fine del progetto. È come una casa: se volete che sia antisismica, deve nascere così. Non potete renderla antisismica una volta costruita senza abbatterla.

3.3.2 Trade-offs e il ruolo dell'Architetto

L'architetto deve bilanciare forze contrastanti provenienti da stakeholder diversi. Le decisioni sono spesso sub-ottimali e prese “al buio” (quando i requisiti sono ancora vaghi).

Solo nelle slide (non spiegato a voce)

Questo dilemma è rappresentato graficamente dalla scelta tra carne “Cotta” o “Cruda”: ogni scelta comporta un vantaggio per uno stakeholder e uno svantaggio per un altro.

3.4 Documentazione: Viste e Punti di Vista

IMPORTANTE PER L'ESAME

Questa è una distinzione fondamentale. Molti studenti confondono i due termini: ricordateli bene per l'esame.

- **Architectural View (Vista):** È la rappresentazione del sistema da una certa prospettiva. È composta da uno o più *modelli* (o diagrammi).
- **Architectural Viewpoint (Punto di Vista):** È l'insieme di regole, convenzioni e linguaggi usati per costruire la vista.

Nota del Prof

Esempio dello stadio: il *Viewpoint* è il concetto teorico di “guardare dalla tribuna stampa” (regole e angolazione). La *View* è la foto specifica che scattate in quello stadio particolare applicando quel punto di vista. In questo corso, Modello e Diagramma sono considerati sinonimi.

3.5 Software Product Lines (SPL)

Una SPL è una famiglia di prodotti che condividono un set comune di funzionalità (**Core Assets**). Si basa sul *Systematic Reuse*.

- **Scoping:** Decidere cosa includere nella piattaforma comune. Se lo scope è troppo largo si sprecono soldi; se è troppo stretto si perde l'opportunità di riuso.
- **Vantaggi:** Riduzione drastica dei tempi e dei costi di sviluppo per i prodotti successivi al primo.

Nota del Prof

Pensate alle auto: la Mercedes usa lo stesso tasto per il finestrino su tutti i modelli. La BMW usa lo stesso motore fisico per vari modelli, limitandone la potenza via software. Nel software il riuso è vitale, ma genera conflitti di budget: il Team A spesso non vuole pagare il costo extra per rendere un modulo riutilizzabile anche dal Team B.

3.6 Stili Architetture

Lo **Stile Architetture** è un pattern astratto; l'**Architettura** è la sua applicazione pratica.

1. **Client/Server:** Il server gestisce dati e sicurezza, i client richiedono servizi. Dissaccoppiamento parziale (il server non conosce i client).
2. **Peer-to-Peer:** Ogni nodo è sia client che server.

Nota del Prof

Come BitTorrent: chi scarica deve anche caricare, evitando colli di bottiglia.

3. **Pipes and Filters:** Stream di dati trasformati in sequenza.

Nota del Prof

Usato in sistemi critici come l'avionica. Performance altissime, riuso quasi nullo.

4. **Shared Data:** Comunicazione tramite un database centrale.

Solo nelle slide (non spiegato a voce)

Si divide in **Repository** (l'agente chiede i dati) e **Blackboard** (il sistema notifica gli agenti quando i dati cambiano).

Ecco il codice LaTeX per la **Lezione 4**, strutturato per essere inserito nel tuo file principale tramite . Ho mantenuto lo stile dei box personalizzati e inserito le citazioni basate sui documenti forniti.

Snippet di codice

Capitolo 4

Debito Tecnico e Code Smells

4.1 Il Debito Tecnico (Technical Debt)

Il Debito Tecnico (TD) rappresenta l'applicazione di concetti finanziari al dominio dell'ingegneria del software, aggiungendo criteri di tempo ed economia al classico concetto di manutenibilità.

IMPORTANTE PER L'ESAME

Il debito tecnico si scompone in due concetti finanziari fondamentali che lo distinguono dalla semplice manutenibilità:

- **Capitale (Principal):** Il costo (effort) necessario per eliminare il debito, ovvero il tempo speso per il refactoring.
- **Interessi (Interest):** La penalità futura pagata se il debito non viene eliminato, che si traduce in rallentamento dello sviluppo o maggiore probabilità di bug.

4.1.1 Cause e Conseguenze

Il debito emerge tipicamente per ottimizzazioni a breve termine che creano handicap a lungo termine. Le cause includono la **crescita organica** (aumento naturale della complessità) e **scelte opportunistiche** (trade-off per rispettare le scadenze).

Nota del Prof

Spesso si creano trade-off tra qualità interna e "Time to Market". Seguendo il principio Agile "Working code over comprehensive documentation", gli sviluppatori possono trascurare la modularizzazione o i commenti pur di consegnare software funzionante e farsi pagare .

IMPORTANTE PER L'ESAME

Il debito tecnico impatta sempre la release successiva: se il codice non è manutenibile oggi, lo sviluppo futuro sarà più lento e pronò ai difetti.

Solo nelle slide (non spiegato a voce)

Gli analizzatori statici (come SonarQube) faticano a calcolare gli "Interessi" (il costo futuro), rendendoli attualmente una metrica poco chiara.

4.1.2 Strategie di Gestione

Nota del Prof

Il debito tecnico "scade": se un modulo di scarso valore non verrà mai più modificato, non produrrà interessi. È un errore fare refactoring su codice che non verrà più toccato.

Le strategie di gestione includono:

- **Monitoraggio e Refactoring:** Uso di strumenti come SonarCloud per tracciare il trend delle violazioni.
- **Quality Gate:** Una regola che blocca automaticamente i commit contenenti nuove violazioni.

Nota del Prof

Metafora della barca bucata: la prima cosa da fare non è svuotare l'acqua (vecchio debito), ma tappare il buco (usare il quality gate per non farne entrare di nuovo).

4.2 Introduzione ai Code Smells

Occorre distinguere tra tre concetti:

- **Quality Rule:** Principio ingegneristico validato (es. commenti, bassa complessità).
- **Violation o Code Smell:** Porzione di codice non conforme a una regola di qualità.
- **Defect (Bug):** Problema che impatta la qualità esterna (es. crash).

IMPORTANTE PER L'ESAME

I Code Smells **non sono bug**. Non causano malfunzionamenti immediati ma rendono la manutenzione difficile. Agiscono come un "campo minato", aumentando la probabilità che uno sviluppatore introduca difetti reali.

4.3 Catalogo dei "Worst Smells"

I "Worst Smells" sono violazioni che non portano alcun vantaggio in nessuna circostanza, a differenza di alcuni smell "classici" (come God Class o Code Clones) che possono nascere da trade-off temporali.

4.3.1 Esempi di Worst Smells

- **Modificatori non ordinati:** Crea confusione visiva inutile.
- **Object.finalize() sovrascritto:** In Java, forzare la distruzione manuale è spesso deleterio.
- **Codice commentato:** Sporca la codebase; spesso sono rimasugli di prototipi.
- **Mancanza del "break" in uno Switch:** Causa comportamenti imprevisti proseguendo nei case successivi.
- **Concatenazione di Stringhe con '+' in un ciclo:** Degrada gravemente le performance; il refactoring corretto prevede l'uso di **StringBuilder**.
- **Pessimo Naming:** Esempi includono attributi chiamati A, B o C.

Nota del Prof

Chiamare un metodo `ThisIsAGreatMethod1` non fa risparmiare tempo significativo: non siamo in Formula 1 dove si risparmiano millisecondi.

Solo nelle slide (non spiegato a voce)

L'analisi su 27 progetti Apache ha rivelato che i Worst Smells non sono meno frequenti né più severi degli altri, ma non avendo ragioni di esistere, vanno bloccati sempre tramite Quality Gates.

Capitolo 5

Machine Learning e Software Analytics per la Software Engineering

5.1 Definizione di Machine Learning, ruolo di statistica e informatica

Il **Machine Learning (ML)** è lo studio degli algoritmi che migliorano le proprie performance in un determinato **task** attraverso l'**esperienza**. L'obiettivo è ottimizzare un criterio di performance utilizzando dati di esempio o esperienze passate.

Questa disciplina unisce due campi fondamentali:

- **Statistica**: necessaria per trarre inferenze a partire da un campione di dati.
- **Informatica**: necessaria per fornire algoritmi efficienti in grado di risolvere il problema di ottimizzazione, oltre a rappresentare e valutare il modello per l'inferenza.

Nota del Prof

È fondamentale comprendere che ogni misurazione e ogni modello di ML, sebbene si basi su dati raccolti nel passato, ha come unico scopo quello di **prendere decisioni sul futuro**. L'intero sistema si fonda su un assioma filosofico implicito ma cruciale: assumiamo che il futuro sia simile al passato. Senza questa assunzione di base, l'uso dei dati storici per addestrare un modello sarebbe completamente inutile.

5.2 Esempi d'uso: associazioni, classificazione, regressione

Il Machine Learning viene applicato a diverse tipologie di problemi.

5.2.1 Apprendimento di associazioni

L'**apprendimento di associazioni** calcola la probabilità che un evento si verifichi in concomitanza di un altro. Un esempio tipico è la **basket analysis**, in cui si stima la probabilità che chi compra un certo prodotto acquisti anche un altro prodotto.

5.2.2 Classificazione

La **classificazione** serve a dividere i dati in categorie. Tra gli esempi più importanti troviamo:

- **Credit scoring**: differenziare clienti a basso o alto rischio in base a reddito e risparmi.
- **Face recognition**.
- **Character recognition**.
- **Speech recognition**.
- **Diagnosi medica**.
- **Predizione del click** su pubblicità web.

5.2.3 Regressione

La **regressione** viene utilizzata quando l'output non è una categoria, ma un valore numerico continuo. Un esempio classico è la predizione del **prezzo di un'auto usata** in base ai suoi attributi.

Nota del Prof

Sistemi come Google o Netflix basano il loro intero modello di business sulla predizione. Google non vende solo spazi pubblicitari, ma predice quale utente cliccherà su quale banner per massimizzare i profitti. Netflix usa algoritmi di classificazione e raccomandazione per permettere all'utente di trovare rapidamente il contenuto desiderato. Inoltre, i sistemi di autenticazione biometrica non cercano una corrispondenza pixel per pixel, ma utilizzano il ML per capire quali variazioni sono significative e quali no.

Le principali applicazioni del ML si articolano quindi in tre grandi famiglie: **associazioni**, **classificazione** e **regressione**. Esse coprono problemi di co-occorrenza, categorizzazione e stima di valori numerici.

5.3 Tipi di apprendimento: supervised, unsupervised, semi-supervised, reinforcement

I modelli di ML possono apprendere in modi diversi a seconda dei dati disponibili.

5.3.1 Supervised learning

Nel **supervised learning**, il modello riceve sia i dati di addestramento sia gli output desiderati, cioè le **label**.

Nota del Prof

Un esempio è fornire al modello la dimensione di un tumore dicendogli esplicitamente se in passato si è rivelato benigno o maligno, affinché impari a classificarlo in futuro.

5.3.2 Unsupervised learning

Nell'**unsupervised learning**, vengono forniti dati di addestramento senza output desiderati. Lo scopo è trovare struttura, similarità o gruppi latenti nei dati.

Nota del Prof

In questo caso l'obiettivo non è predire un valore esatto, ma organizzare i dati in cluster simili tra loro, ad esempio per raggruppare geni simili o fare network analysis.

5.3.3 Semi-supervised learning

Solo nelle slide (non spiegato a voce)

Il **semi-supervised learning** utilizza dati di addestramento in cui solo una parte limitata delle istanze possiede le label corrette.

5.3.4 Reinforcement learning

Solo nelle slide (non spiegato a voce)

Il **reinforcement learning** è una tipologia di apprendimento basata su un sistema di ricompense e punizioni. Nelle slide è citato, ma non viene approfondito a voce.

Nel complesso, la distinzione tra questi paradigmi dipende dal tipo di supervisione disponibile e dal modo in cui il sistema riceve informazione utile per apprendere.

5.4 Etica del data mining e del ML

L'utilizzo dei dati solleva questioni etiche rilevanti. L'anonimizzazione dei dati è difficile, e l'uso di certe informazioni può condurre a discriminazioni. Per esempio, utilizzare razza, sesso o religione per l'approvazione di un prestito è eticamente problematico, mentre in ambito medico gli stessi dati possono essere legittimi.

Le domande fondamentali sono:

- Chi ha accesso ai dati?
- Per quale scopo sono stati raccolti?
- Quali conclusioni possono essere tratte legittimamente?

Nota del Prof

Quando un sistema ML prende decisioni, la responsabilità è un tema complesso. Se una Tesla a guida autonoma causa un incidente, oggi la responsabilità legale ricade ancora su chi è al volante, anche in virtù degli accordi di licenza accettati dagli utenti. Inoltre, il ML tende a replicare i bias umani presenti nei dati storici: se nel passato una certa etnia o un certo genere ha subito discriminazioni, il modello può apprendere e riprodurre quel comportamento in futuro.

L'etica del ML impone quindi attenzione a **privacy**, **bias**, **discriminazione** e **responsabilità**. I dati storici non sono neutrali, e i modelli possono ereditare le distorsioni del contesto da cui provengono.

5.5 Terminologia: istanze, feature, attributi nominali/numerici, output, modelli

Per utilizzare correttamente gli algoritmi di ML è necessario padroneggiare una terminologia standard.

5.5.1 Instances

Le **instances** sono i singoli esempi indipendenti del concetto da apprendere.

Nota del Prof

In un dataset strutturato a tabella, l'istanza rappresenta la singola riga, ad esempio la combinazione di una specifica classe software in una specifica release.

5.5.2 Features o attributi

Le **feature** sono gli aspetti misurabili di un'istanza. Possono essere:

- **Nominali**: valori simbolici o categorici.
- **Numerici**: valori quantitativi su cui si possono eseguire operazioni matematiche.

Nota del Prof

Le feature rappresentano le colonne della tabella. In genere, l'ultima colonna è l'output o variabile da predire, mentre le altre sono metriche che aiutano nella predizione.

5.5.3 Output e modelli

La conoscenza prodotta da un sistema di ML può essere rappresentata tramite:

- **tabelle decisionali**;
- **modelli lineari**;

- alberi decisionali;
- regole;
- cluster.

IMPORTANTE PER L'ESAME

Molti ambienti di ML, in particolare **Weka**, non supportano correttamente gli attributi con scala ordinale, ad esempio *Low*, *Medium*, *High*. Weka tende a trattarli come semplici categorie nominali, perdendo l'informazione dell'ordine. Per questo motivo, spesso occorre eliminare la colonna oppure trasformarla in una variabile binaria, accettando il relativo trade-off nella perdita di informazione.

Un dataset è dunque una tabella in cui le **righe** rappresentano le istanze e le **colonne** rappresentano le feature. La corretta interpretazione del tipo di attributo è essenziale, soprattutto quando si usano strumenti come Weka.

5.6 Software quality e costo dei bug

La **qualità del software** ha un impatto economico enorme. I bug costano a livello globale migliaia di miliardi di dollari, e le attività di **Quality Assurance (QA)** risultano estremamente costose.

Nei sistemi industriali di grandi dimensioni:

- vengono effettuate migliaia di **code review** al giorno;
- vengono eseguiti milioni di **test**;
- ogni modifica deve essere controllata con attenzione.

Nota del Prof

Poiché il testing è molto oneroso, diventa fondamentale stimare in anticipo quali entità del codice, come classi o metodi, abbiano maggiore probabilità di essere difettose, così da concentrare lì l'effort di QA.

Ne segue che la defect prediction nasce anche come risposta economica e organizzativa al problema della qualità: non è sostenibile controllare tutto con la stessa intensità, quindi occorre **prioritizzare**.

5.7 Software analytics: definizione, obiettivi e fonti dati

La **Software Analytics** è la combinazione di **Software Data** e **Data Analytics**. Il software produce una grande quantità di dati distribuiti tra repository, sistemi di build, log di CI/CD, issue tracker, code review e file di configurazione.

Gli obiettivi principali della Software Analytics sono:

1. **Quality improvement**: prevenire crash e bug.

2. **Process improvement:** capire l'impatto di pratiche come code review e rapid release.
3. **Productivity improvement:** valutare l'effetto della CI e di altre pratiche sul team.
4. **Empirical theory building:** costruire teorie empiriche sull'ingegneria del software.

La Software Analytics sfrutta quindi i dati prodotti dagli strumenti di sviluppo per migliorare **qualità**, **processi**, **produttività** e comprensione empirica del software.

5.8 AI/ML applicati alla Software Engineering

L'Intelligenza Artificiale sta trasformando l'Ingegneria del Software.

5.8.1 Applicazioni alla qualità

Tra le applicazioni principali ci sono:

- **defect prediction;**
- **vulnerability prediction;**
- **malware prediction;**
- **test case generation.**

5.8.2 Applicazioni alla produttività

L'AI può supportare anche la produttività del team, ad esempio tramite:

- generazione di **UI**, requisiti, codice e commenti;
- previsione della **produttività** di sviluppatori o team;
- **recommendation** di developer o reviewer;
- identificazione del **turnover** degli sviluppatori.

Solo nelle slide (non spiegato a voce)

Nelle slide vengono citati esempi industriali concreti, come un **AI coding assistant** sviluppato in collaborazione con Mozilla, i tool **SapFix** e **Sapienz** di Facebook per trovare e correggere bug automaticamente, e **Sketch2Code** di Microsoft per trasformare schizzi su lavagna in codice HTML.

Nel contesto della Software Engineering, l'AI non serve quindi solo a prevedere bug, ma anche a supportare gli sviluppatori nella produzione di codice, nel testing e nella gestione del lavoro di squadra.

5.9 Defect prediction e ML for software defects

Nel contesto dei difetti software, il ML ha tre grandi obiettivi:

1. **Predire** i difetti futuri, per ottimizzare le risorse di QA.
2. **Spiegare** perché un software fallisce.
3. **Costruire** teorie empiriche sulla defectiveness.

Nota del Prof

Prevenire è meglio che curare. Eliminare code smells e individuare le classi più rischiose consente di combattere i bug in modo proattivo e di stabilire priorità di testing più efficaci.

La defect prediction non si limita quindi a indicare *dove* è probabile trovare un difetto, ma cerca anche di chiarire *perché* quel rischio esiste e come ridurlo a livello di processo o di codice.

5.10 Pipeline: measure, clean data, analyze, model, explain

L'applicazione della Software Analytics alla defect prediction segue una pipeline precisa:

1. **Measure**: estrazione dei dati grezzi e raccolta delle metriche.
2. **Clean Data**: pulizia dei dati.
3. **Analyze**: ricerca di correlazioni e relazioni tra feature.
4. **Model**: costruzione e addestramento di modelli analitici o black-box.
5. **Explain**: estrazione di conoscenza utile per spiegare le predizioni.

Nota del Prof

Nel corso verrà seguita esattamente questa pipeline: si partirà da una tabella grezza, la si analizzerà, si sceglieranno e si normalizzeranno le colonne, si costruirà il modello e infine si cercherà di estrarre conoscenza utile a evitare in futuro gli stessi errori.

In sintesi

La pipeline consente di trasformare dati grezzi provenienti dai repository in **modelli predittivi** e poi in **conoscenza spiegabile e azionabile**. È uno schema concettuale fondamentale perché collega raccolta dati, modellazione e interpretazione.

5.11 Estrazione dati da ITS e VCS e raccolta metriche

Il primo passo della pipeline consiste nell'estrazione dei dati da due sorgenti principali:

- **ITS (Issue Tracking System)**: ad esempio Jira, che contiene ticket, bug report e informazioni di progetto.
- **VCS (Version Control System)**: ad esempio Git, che contiene codice sorgente, snapshot e log dei commit.

Da queste fonti si raccolgono diverse famiglie di metriche:

- **Code Metrics**: dimensione, complessità, design object-oriented, accoppiamento, coesione.
- **Process Metrics**: numero di commit, churn, difetti pre-release, pratiche di sviluppo.
- **Human Factors**: code ownership, numero di sviluppatori che hanno toccato il file, esperienza dello sviluppatore.

Nota del Prof

Più persone modificano una stessa classe, maggiore è il rischio di introdurre difetti. Inoltre, se un commit è effettuato da uno sviluppatore che non ha esperienza su quella classe, la probabilità di errore cresce sensibilmente.

Combinando dati da **Jira** e **Git**, si ottengono quindi metriche sul codice, sul processo e sui fattori umani, tutte utili per la defect prediction.

5.12 Identificazione difetti, labeling e defect dataset

Per addestrare un modello di defect prediction è necessario costruire un dataset etichettato, in cui ogni classe venga marcata come **difettosa** o **non difettosa**.

Questo processo dipende dal **linkage** tra:

- ticket nell'Issue Tracking System;
- commit nel Version Control System;
- versioni del software coinvolte nel ciclo di vita del difetto.

Nota del Prof

Quando uno sviluppatore effettua un commit per risolvere un problema, inserisce spesso nel messaggio del commit l'ID del ticket Jira associato. Questo ID rappresenta il razionale del cambiamento. Grazie a questa connessione tra commit e ticket, e tramite tecniche come **SZZ** e **Proportion**, è possibile non solo sapere quale commit ha corretto il bug, ma anche risalire a ritroso fino al commit che lo aveva introdotto.

In sintesi

Il **labeling** dipende dalla capacità di collegare correttamente ticket, commit e release. Questo collegamento è alla base della costruzione del **defect dataset** ed è il passaggio che determina la qualità dei dati su cui verranno addestrati i modelli.

5.13 Black-box models e necessità di spiegazione

Molti algoritmi di apprendimento producono modelli **black-box**: ricevono un input e restituiscono una predizione, ma senza rendere trasparente il ragionamento interno che ha portato a quella decisione.

Per esempio, un modello può dire che una certa classe ha alta probabilità di essere difettosa, ma senza spiegare quali fattori abbiano portato a quel risultato.

Nota del Prof

Un predittore come una rete neurale può essere molto accurato, ma non sa giustificare in modo leggibile il perché della sua scelta. Nell'ingegneria del software, invece, capire il motivo della difettosità è cruciale, perché consente al manager di agire sulla causa e non soltanto sul sintomo.

In contesto ingegneristico, quindi, la sola accuratezza non basta: il valore operativo di una predizione aumenta notevolmente quando essa è accompagnata da una spiegazione comprensibile.

5.14 XAI, GDPR, fairness, accountability, transparency

La necessità di spiegare i modelli è legata sia a motivazioni tecniche sia a motivazioni etiche e legali. In questo contesto si colloca il paradigma **FAT**:

- **Fairness**;
- **Accountability**;
- **Transparency**.

L'**Articolo 22 del GDPR** richiede che i processi decisionali automatizzati che hanno impatto sugli individui siano accompagnati da una spiegazione adeguata.

Questo porta a domande fondamentali:

- I sistemi di AI rispettano le leggi?
- I modelli producono decisioni discriminatorie?
- Comprendiamo davvero perché un modello fa una certa predizione?

Nota del Prof

Una tecnica di Explainable AI consiste nell'affiancare a un modello black-box, come una rete neurale, un modello gemello spiegabile, ad esempio un albero decisionale.

Invece di addestrare l'albero direttamente sui dati originali, lo si addestra sugli output generati dalla rete neurale, così da mimarne il comportamento e renderne più leggibili le decisioni.

In sintesi

La **XAI** nasce dall'esigenza di rendere i modelli comprensibili, verificabili e compatibili con vincoli legali ed etici come quelli imposti dal GDPR. In questo quadro, la spiegabilità non è un'aggiunta opzionale, ma una proprietà sempre più centrale.

5.15 CMMI Level 5 e process improvement

Il **CMMI (Capability Maturity Model Integrated)** è uno standard per misurare la maturità dei processi di sviluppo software.

I livelli principali sono:

1. **Initial**
2. **Managed**
3. **Defined**
4. **Quantitatively Managed**
5. **Optimizing**

Il **Level 5**, cioè *Optimizing*, rappresenta il livello più elevato: le performance del processo vengono migliorate in modo continuo tramite innovazione e controllo quantitativo.

Nota del Prof

Il professore riporta un'esperienza diretta in cui ha aiutato un'azienda a mantenere il Livello 5 del CMMI. L'obiettivo era dimostrare statisticamente che certe strategie di miglioramento avessero prodotto benefici reali. I risultati hanno mostrato una forte riduzione dei difetti e un marcato aumento del profitto per dipendente.

Il riferimento al CMMI mostra bene come metriche, controllo statistico e miglioramento continuo possano essere integrati in una visione strutturata della qualità.

5.16 Process chart e controllo statistico

Per controllare la stabilità del processo si utilizzano i **process chart** o **control chart**. Essi servono a:

- identificare problemi mentre si verificano;
- predire l'intervallo atteso dei risultati del processo;
- verificare se il processo è statisticamente stabile;
- decidere se intervenire con correzioni locali o con cambiamenti più strutturali.

La costruzione di un process chart prevede:

1. scelta della variabile da osservare;
2. scelta dell'unità temporale;
3. calcolo della **media**;
4. calcolo dell'**Upper Control Limit** come media più tre deviazioni standard;
5. calcolo del **Lower Control Limit** come media meno tre deviazioni standard.

Nota del Prof

Finché i dati restano all'interno dei limiti di controllo, il processo è considerato statisticamente stabile. Se invece un punto supera uno dei limiti, si attiva una retrospettiva per analizzare la root cause dell'anomalia e capire come evitare che si ripeta.

I **process chart** permettono dunque di monitorare il processo nel tempo e di individuare rapidamente deviazioni anomale che richiedono analisi e intervento.

5.17 Casi applicativi mostrati nelle slide, anche se solo introdotti

Le lezioni presentano anche alcuni casi applicativi concreti.

5.17.1 Simulatore di release e defect prediction

Nota del Prof

È stata sviluppata un'interfaccia per supportare i manager nel **release planning**. Modificando alcuni slider relativi a variabili di input, come la percentuale di God Classes, la compliance architetturale o il numero di nuovi sviluppatori, il modello ricalcolava dinamicamente il numero atteso di difetti nella release successiva. Il sistema forniva anche intervalli di confidenza e scenari di best case e worst case.

5.17.2 Traceability recovery

Solo nelle slide (non spiegato a voce)

Nelle slide si introducono studi sull'uso di tecniche NLP e classificatori di ML per stimare il numero di link rimanenti nel recupero della tracciabilità dei requisiti.

5.17.3 Defectiveness prediction via JIT

Solo nelle slide (non spiegato a voce)

Le slide finali introducono il tema del miglioramento della defect prediction combinando informazioni **Just-In-Time** a livello di commit con metriche classiche a livello di classe e metodo.

Le tecniche discusse trovano applicazione sia in strumenti concreti di supporto decisionale per i manager, sia in linee di ricerca avanzate su **tracciabilità** e **predizione dei difetti**.

5.18 Riepilogo finale

I concetti chiave emersi sono i seguenti:

- Il **Machine Learning** apprende dai dati passati per prendere decisioni sul futuro.
- Le principali famiglie di problemi sono **associazioni**, **classificazione** e **regressione**.
- La **Software Analytics** usa dati provenienti da repository, issue tracker e pipeline di sviluppo per migliorare qualità, processi e produttività.
- La **defect prediction** serve a stimare quali classi o componenti sono più a rischio di contenere difetti.
- La pipeline fondamentale è: **Measure** → **Clean Data** → **Analyze** → **Model** → **Explain**.
- Il **labeling** delle classi dipende dal corretto collegamento tra ticket, commit e release.
- I modelli **black-box** sono potenti ma poco trasparenti, perciò diventa centrale la **Explainable AI**.
- La spiegabilità è importante non solo dal punto di vista tecnico, ma anche per ragioni di **fairness**, **accountability**, **transparency** e conformità al **GDPR**.
- Il **CMMI Level 5** e i **process chart** mostrano come il controllo quantitativo del processo possa tradursi in miglioramenti concreti di qualità e produttività.

Capitolo 6

SZZ: Identifying Buggy Entities

6.1 Contesto

6.1.1 Cos'è la defect prediction

La **defect prediction** è una tecnica che mira a identificare gli **artefatti software** (come classi, metodi o linee di codice) che hanno un'alta probabilità di presentare un difetto. L'obiettivo principale è **ridurre i costi e i tempi del testing e della code review**, permettendo agli sviluppatori di concentrarsi sulle parti più critiche del codice.

Nota del Prof

Il testing, sia esso automatico o manuale (come l'ispezione del codice), può essere molto oneroso in termini economici e computazionali. Ottimizzare questi sforzi è il vero scopo di queste predizioni. Anche se per semplicità nel progetto si lavora a livello di classe, questi concetti si applicano a vari livelli di granularità, come linea di codice, commit, metodo e classe.

6.1.2 Perché la qualità del dataset è fondamentale

L'**affidabilità di un modello di predizione** dipende direttamente dalla **qualità dei dati** usati per addestrarlo.

Nota del Prof

Vale il principio del *Garbage In, Garbage Out*: se i dati in input contengono impurità o errori, anche l'accuratezza del modello ne risentirà proporzionalmente.

6.1.3 Principali fonti di rumore nei dataset

Le principali **fonti di rumore** indicate sono:

- defect misclassification;
- defect origin.

Solo nelle slide (non spiegato a voce)

Lavori precedenti hanno identificato queste fonti di rumore e hanno proposto delle soluzioni per gestirle.

Nota del Prof

Un esempio pratico di rumore è che circa un terzo dei ticket classificati come “bug” in realtà sono nuove “feature” (*misclassification*). Un'altra fonte di rumore è il **linkage**: se nei commenti dei commit non viene riportato l'ID del ticket (ad esempio di Jira), si perde lo storico delle classi difettose, generando **falsi negativi** nel dataset.

6.2 Cos'è SZZ

6.2.1 Definizione e obiettivo

SZZ è un processo algoritmico usato per identificare il **momento esatto in cui un difetto è stato introdotto** all'interno di un progetto software.

Nota del Prof

Il nome dell'algoritmo deriva semplicemente dai cognomi dei suoi autori.

6.2.2 Uso del versioning e dell'annotation

Il metodo sfrutta il **meccanismo di annotazione** dei sistemi di versionamento, come per esempio il comando `git blame` in Git.

Nota del Prof

L'annotazione permette di analizzare una riga di codice nel suo stato attuale e scoprire in quale revisione precedente è stata modificata l'ultima volta e da chi.

6.2.3 Idea di partire dal fix per risalire al bug-introducing change

L'approccio cerca di determinare, per le specifiche **linee di codice modificate durante la correzione di un difetto**, quando queste erano state alterate per l'ultima volta prima del fix.

IMPORTANTE PER L'ESAME

Il concetto chiave di SZZ è definire come “sorgente” del difetto la stessa esatta zona di codice impattata dalla sua correzione. Se per riparare la macchina il meccanico cambia la trasmissione, il problema era la trasmissione; allo stesso modo, se nel fix di un bug viene modificata una riga di codice, è ragionevole assumere che l'errore risiedesse proprio in quella riga.

6.3 Come funziona SZZ

6.3.1 Identificazione del fix commit

Il primo passo consiste nell'identificare il **fix commit**, cioè il commit che corregge il difetto.

Nota del Prof

Si parte dal sistema di tracciamento, ad esempio Jira, si prendono tutti i ticket chiusi e classificati come "bug". Ogni commit associato a questi ticket rappresenta una **fix** nel codice sorgente.

6.3.2 Analisi delle linee modificate

Una volta trovato il commit di fix, si analizzano le **esatte linee di codice toccate** dalla correzione.

Nota del Prof

Si sa con certezza che in quel momento, prima del fix, in quelle righe era presente un bug.

6.3.3 Ricerca all'indietro dell'ultima modifica precedente

Dopo aver individuato le righe corrette, si usa `git blame` per effettuare una **backward search**, cioè una ricerca all'indietro del commit precedente che aveva inserito o modificato quelle stesse righe.

IMPORTANTE PER L'ESAME

L'entità toccata dal fix era difettosa; SZZ individua la prima modifica precedente a quel fix per etichettare il momento in cui l'errore è nato.

6.3.4 Differenza tra bug-fix change e bug-introducing change

È fondamentale distinguere tra:

- **bug-fix change**: il commit che risolve il problema correggendo il codice;
- **bug-introducing change**: il commit passato che aveva materialmente scritto o alterato in modo errato le linee poi corrette dal fix.

6.4 Esempi nelle slide

6.4.1 Esempio 1: “When Do Changes Induce Fixes?”

Solo nelle slide (non spiegato a voce)

Il primo diagramma mostra una sequenza temporale in cui viene segnalato un bug con ID 42233. Successivamente avviene un fix, nella revisione 1.18, in cui vengono modificati tre metodi: `a()`, `b()` e `c()`. L'algoritmo SZZ traccia all'indietro ciascun metodo per trovare il **bug-introducing change**, scoprendo che `a()` era stato modificato nella revisione 1.11, `b()` nella 1.14 e `c()` nella 1.16.

6.4.2 Esempio 2: “The Impact of Refactoring Changes on the SZZ Algorithm: An Empirical Study”

Nota del Prof

Viene illustrato il caso di un ciclo `for`. Nella versione che introduce il bug (V2), la condizione era scritta come `for(int i=0; i<=4; i++)`. In seguito a un'eccezione riportata nel Bug #123 (V3), il fix altera il ciclo facendolo diventare `for(int i=0; i<4; i++)`.

I diagrammi mostrano operativamente i tre passaggi dell'algoritmo:

1. recuperare il **link nel commit di fix** tramite l'ID del bug;
2. eseguire il **diff** con la versione precedente per isolare la riga corretta;
3. tracciare all'indietro quella specifica riga per individuare il **bug-introducing change**.

Nel caso mostrato, il cambiamento difettoso è la presenza della condizione `i<=4`, poi corretta in `i<4`.

6.5 Limiti di SZZ

Le slide elencano diverse **limitazioni** che riducono l'accuratezza del metodo.

6.5.1 Code additions

Solo nelle slide (non spiegato a voce)

Appena il 26–29% dei **bug-introducing commits** può essere trovato usando l'algoritmo basato sull'assunzione che “un bug è stato introdotto dalle linee di codice che sono state modificate per fixarlo”.

Questo limite riduce l'accuratezza perché, se il fix consiste esclusivamente nell'**aggiunta di nuove righe di codice mancanti**, non esiste una modifica precedente da tracciare all'indietro.

6.5.2 Regressive bugs

Nota del Prof

Sono bug che si creano mentre si sta cercando di implementare qualcos'altro. Questo riduce l'accuratezza perché il codice originario, ad esempio `i<=4`, poteva essere corretto in precedenza e diventare sbagliato solo dopo l'introduzione di un'altra feature. SZZ, facendo la ricerca all'indietro, incolpa ingiustamente lo sviluppatore che aveva scritto quella riga in origine, generando un **falso positivo**.

6.5.3 Refactoring

Nota del Prof

Il **refactoring** è una modifica del codice che ne cambia la struttura ma lascia intatta la funzionalità, ad esempio rinominare una variabile. Se la prima modifica precedente trovata da SZZ è un mero refactoring estetico, l'algoritmo attribuirà erroneamente a quel refactoring l'introduzione del bug, perdendo lo storico reale della funzionalità difettosa.

6.5.4 Imprecise backward search

Nota del Prof

SZZ è molto forte nell'identificare **cosa** è difettoso, ma è impreciso nell'identificare **da quanto tempo** lo è. Un codice poteva essere corretto rispetto a requisiti passati e diventare difettoso solo in seguito a un cambiamento dei requisiti. Se si risale troppo indietro, si etichettano come buggy versioni del codice che, al tempo, erano corrette.

6.5.5 Snoring

Solo nelle slide (non spiegato a voce)

Lo **snoring** è solo nominato nell'elenco puntato della slide senza alcuna descrizione.

Nota del Prof

Il professore lo cita esplicitamente chiarendo che è un concetto legato a come la misurazione del dataset cambia nel tempo, ma rimanda la spiegazione dettagliata alle lezioni successive. In questa lezione, quindi, non viene approfondito.

6.6 Da ricordare per l'esame

- **Definizione di SZZ:** algoritmo che sfrutta l'annotazione dei sistemi di versionamento per identificare il commit in cui un difetto è stato introdotto nel codice sorgente.

- **Passaggi essenziali:** identificare i commit di fix, analizzare le linee modificate da questi fix e ricercare all'indietro, tramite `git blame`, la modifica immediatamente precedente su quelle stesse righe.
- **Differenza tra fix commit e bug-introducing commit:** il primo ripara il difetto, il secondo è la revisione precedente che aveva inserito o alterato in modo errato la riga poi corretta.
- **Principali limiti:** *code additions, regressive bugs, refactoring e imprecise backward search.*

Capitolo 7

Leveraging Defects Lifecycle for Labeling Defective Classes

7.1 Contesto e obiettivo del lavoro

7.1.1 Defect prediction

La **defect prediction** ha l'obiettivo di identificare gli artefatti software, ad esempio classi o file, che hanno un'alta probabilità di manifestare un difetto. Lo scopo principale è ridurre i costi di testing e di code review, permettendo agli sviluppatori di concentrare gli sforzi sulle parti più critiche del codice.

7.1.2 Importanza del dataset

Affinché un modello di predizione sia affidabile, è fondamentale costruire **dataset di difetti di alta qualità**. Modelli addestrati su dati rumorosi raggiungono performance sensibilmente peggiori rispetto a modelli addestrati su dati puliti, in particolare in termini di **Recall**.

7.1.3 Labeling delle classi difettose

Nota del Prof

Ci troviamo nella fase iniziale di **misurazione** e creazione del dataset. Per costruire un modello di Machine Learning, dobbiamo prima etichettare correttamente quali classi siano state effettivamente difettose in passato.

7.1.4 Obiettivo generale dello studio

Lo studio ha tre obiettivi principali:

1. misurare la proporzione di difetti per cui è possibile usare un approccio realistico basato sui dati esatti di Jira;
2. proporre un nuovo metodo automatizzato, chiamato **Proportion**, per recuperare l'origine del difetto quando i dati di Jira mancano;

3. confrontare l'accuratezza del nuovo metodo rispetto alle tradizionali implementazioni di **SZZ**.

7.2 Concetti fondamentali

7.2.1 SZZ

SZZ è l'algoritmo classico per identificare l'origine di un difetto nel codice.

7.2.2 Earliest AV in JIRA

L'approccio **Earliest AV in JIRA** consiste nell'usare la prima **Affected Version** riportata manualmente dagli sviluppatori nel sistema di tracciamento, ad esempio Jira.

7.2.3 Defect lifecycle

Ogni difetto ha un **ciclo di vita** che attraversa diverse versioni o release del software.

7.2.4 Significato di IV, OV, FV e AV

- **IV (Introduction / Injected Version):**

Nota del Prof

è la prima versione del codice in cui il difetto è stato fisicamente scritto o iniettato.

- **OV (Opening Version):** è la versione in cui viene creato il ticket di segnalazione.

Nota del Prof

È il momento in cui la *failure*, cioè il malfunzionamento, viene osservata dall'utente in produzione.

- **FV (Fixed Version):** è la prima versione rilasciata dopo il *fix commit*.

Nota del Prof

È la prima versione del software che non contiene più il bug.

- **AV (Affected Versions):** sono tutte le versioni affette dal difetto, comprese nell'intervallo $[IV, FV)$.

7.3 SZZ: funzionamento e limiti

7.3.1 Come funziona SZZ

SZZ individua i commit che hanno introdotto il bug tracciando la storia del codice sorgente.

7.3.2 Uso di git blame / annotation mechanism

SZZ prende le linee di codice modificate da un **fix commit**, cioè il commit che risolve il bug, e utilizza `git blame` per risalire indietro nel tempo e scoprire quando quelle specifiche linee rimosse o modificate erano state introdotte originariamente.

7.3.3 Bug introducing commit

Nota del Prof

Il commit che ha inserito quella riga originaria viene considerato il **bug introducing commit**, e da quel momento la classe viene etichettata come difettosa.

7.3.4 Limiti del metodo

Le ricerche, e il paper stesso, mostrano che SZZ è lontano dall'essere ideale:

- non è perfettamente accurato;
- non riesce a identificare correttamente i **regression bugs**;
- non funziona quando un difetto viene corretto soltanto con **aggiunta di codice**, senza cancellazione o modifica di righe precedenti, perché non ha linee su cui applicare `git blame`.

Nota del Prof

SZZ genera numerosi **falsi positivi**, quando va troppo indietro nel tempo attribuendo la colpa a linee di codice storicamente corrette, e **falsi negativi**, quando invece non risale abbastanza indietro e non individua il vero punto di introduzione del bug.

7.4 Aim dello studio e Research Questions

7.4.1 Idea di stable life-cycle dei defect

L'intuizione centrale dello studio è che i difetti abbiano un **ciclo di vita stabile**. Esisterebbe cioè una proporzione relativamente costante tra il numero di versioni necessarie per accorgersi di un bug e quelle necessarie per risolverlo.

7.4.2 Research Questions

Le domande di ricerca sono:

- **RQ1:** le **Affected Versions (AV)** sono disponibili e consistenti in Jira?
- **RQ2:** i vari metodi, ad esempio SZZ e Proportion, hanno accuratèzze diverse nell'etichettare le **Affected Versions**?

- **RQ3:** i metodi hanno accuratezze diverse nell'etichettare le **singole classi difettose**?
- **RQ4:** l'uso di metodi diversi porta all'identificazione di **feature importanti** diverse nei modelli di predizione?

7.5 RQ1: Is the AV available and consistent?

7.5.1 Rationale

Si vuole verificare se gli sviluppatori inseriscano regolarmente il campo **Affected Version** in Jira e, nel caso in cui lo facciano, se quel dato sia **consistente**, cioè compatibile con la sequenza temporale delle versioni.

Nota del Prof

Se il bug risulta riportato in una versione futura rispetto all'apertura del ticket, allora il campo è stato compilato in modo errato.

7.5.2 Design e selezione di progetti e bug

Sono stati analizzati **212 progetti Apache** tracciati sia su Git sia su Jira. Sono stati selezionati i ticket che rispettavano i seguenti criteri:

- `Type == "Bug";`
- `Status == "Closed" oppure "Resolved";`
- `Resolution == "Fixed".`

Sono stati esclusi:

- i bug senza un commit di fix collegato su Git;
- i difetti **non post-release**, cioè risolti prima del rilascio in produzione.

7.5.3 Risultati

Su un totale di **125.860 bug validi**, ben **63.539 bug**, cioè il **51%**, non avevano un campo AV utilizzabile oppure presentavano un'AV inconsistente.

7.5.4 Discussione e implicazioni pratiche

Nella maggior parte dei progetti esaminati esiste più del **25%** di difetti senza AV affidabile. Ne segue che l'approccio realistico, basato esclusivamente sulle informazioni di Jira, è **inapplicabile** nella maggioranza dei casi.

Nota del Prof

Se non abbiamo l'AV e non usiamo algoritmi compensativi, perdiamo statisticamente una parte rilevante delle classi difettose, generando molti **falsi negativi**.

7.6 RQ2: Do methods have different accuracy in AV labeling?

7.6.1 Razionale

Dal momento che in RQ1 si osserva che l'AV spesso manca, occorre valutare l'accuratezza di diverse euristiche per calcolarla automaticamente.

7.6.2 Formula della Proportion e significato di P

La costante proporzionale viene definita come:

$$P = \frac{FV - IV}{FV - OV}$$

La formula per predire l'inizio del bug quando manca l'AV è:

$$\text{Predicted IV} = FV - (FV - OV) \cdot P$$

Nota del Prof

L'intuizione è simile a quella delle proporzioni corporee: anche se gli individui hanno altezze diverse, certe relazioni interne tendono a restare stabili. Allo stesso modo, difetti diversi possono avere durate diverse, ma la proporzione tra tempo di scoperta e tempo di correzione tende a mantenersi relativamente costante.

7.6.3 Varianti di Proportion

- **Cold Start:** calcola P come mediana dei valori osservati in tutti gli altri progetti.
- **Increment:** calcola P come media dei difetti corretti nelle versioni precedenti dello stesso progetto.
- **Moving Window:** calcola P come media dell'ultimo 1% dei difetti corretti, risultando più robusto rispetto a outlier e cambiamenti storici di processo.

Nota del Prof

A lezione è stata discussa anche una variante chiamata **Total**, non presente nel paper, che calcola P su tutti i difetti passati e futuri.

IMPORTANTE PER L'ESAME

La variante **Total** è concettualmente sbagliata nel Machine Learning perché viola il **realismo temporale**. Usare dati del futuro per etichettare il passato porta a una sovrastima artificiale delle performance del classificatore. In un contesto reale, il modello non avrebbe mai accesso ai bug futuri per stimare P .

7.6.4 Metodi confrontati

I metodi messi a confronto sono:

- **Simple**, che assume $IV = OV$;
- **SZZ_B** (Basic);
- **SZZ_U** (Unleashed);
- **SZZ_RA** (Refactoring Aware);
- i tre metodi **Proportion**;
- le versioni **SZZ_X+**, che combinano SZZ con i risultati di Simple.

7.6.5 Metriche

Le metriche considerate sono:

- **TP, TN, FP, FN**;
- **Precision**;
- **Recall**;
- **F1**;
- **Kappa**;
- **MCC** (*Matthews Correlation Coefficient*).

IMPORTANTE PER L'ESAME

Il significato pratico di **FP** e **FN** è centrale. Nel contesto software:

- **Falso Negativo (FN)**: il sistema dice che la classe non ha bug, ma in realtà lo ha. È il caso più pericoloso, perché la classe non viene testata e il difetto può arrivare in produzione.
- **Falso Positivo (FP)**: il sistema dice che la classe ha un bug, ma in realtà non ce l'ha. Questo comporta uno spreco di effort di testing, ma non introduce direttamente un bug in produzione.

7.6.6 Risultati e discussione

Dai risultati emerge che:

- tutti i metodi **Proportion** hanno **Precision** e accuratezza composita (**F1, MCC, Kappa**) superiori rispetto a tutti i metodi SZZ;
- **SZZ_U** ottiene la **Recall** più alta in assoluto;
- **SZZ_B+** ha la migliore Precision e le migliori metriche composite all'interno della famiglia SZZ.

Solo nelle slide (non spiegato a voce)

L'incremento di accuratezza ottenuto combinando *SZZ_X* con **Simple**, generando *SZZ_X+*, è statisticamente significativo secondo il *Wilcoxon Signed Rank test* con $p < 0.01$, ma praticamente irrilevante, poiché l'aumento osservato è inferiore all'1%.

7.7 RQ3: Do methods have different accuracy in classes labeling?

7.7.1 Differenza rispetto a RQ2

In RQ2 si misura la validità dell'intervallo temporale delle **Affected Versions**; in RQ3, invece, si passa a una granularità più fine, cioè alla **singola coppia classe-versione**.

7.7.2 Design

Si usano gli stessi metodi e le stesse metriche di RQ2, ma una coppia **classe-release** viene etichettata come difettosa se almeno un **defect-fix commit** ha toccato quella classe in quella specifica versione.

7.7.3 Risultati

I risultati mostrano che:

- anche a livello class-level i metodi **Proportion** ottengono **Precision** e metriche composite migliori rispetto a **SZZ**;
- **SZZ_U** mantiene la **Recall** più alta;
- **Proportion_MovingWindow** domina nettamente tutti i metodi su tutte le metriche composite.

7.7.4 Discussione

Tutti i metodi risultano più accurati nel predire le classi rispetto alle AV. Questo accade perché una singola classe può essere affetta da più difetti simultaneamente: anche se un metodo manca un difetto, può comunque azzeccare l'etichetta grazie a un altro difetto concomitante.

L'incremento mediano osservato passando da AV labeling a class labeling è:

- +13% in **Precision**;
- +16% in **F1**;
- +39% in **Kappa**.

7.8 RQ4: Do methods lead to different identification of important features?

7.8.1 Razionale

RQ4 valuta l'effetto a cascata del labeling. Se il dataset viene etichettato male, allora anche le **feature** selezionate per addestrare il modello potrebbero risultare sbagliate, così come le loro correlazioni con la defectiveness.

7.8.2 Feature selection

Viene usato un approccio di **Exhaustive Search Feature Selection**, tramite Weka e `CfsSubsetEval`, che valuta i sottoinsiemi di feature privilegiando quelle altamente correlate con il difetto e poco inter-correlate tra loro.

7.8.3 Correlation analysis e Standard Mean Relative Error

Per valutare la bontà delle correlazioni si misura la discrepanza tra:

- la correlazione osservata nel dataset **vero**;
- la correlazione osservata nel dataset **stimato** dai vari metodi.

L'errore è definito come:

$$\text{Errore} = \frac{|\text{Predicted} - \text{Actual}|}{\text{Actual}}$$

7.8.4 Risultati

I metodi **Proportion** risultano più accurati dei metodi SZZ in tutte e cinque le metriche considerate. In particolare, mostrano **Precision** e **Recall** mediane perfette, pari al 100%. L'errore dei metodi Proportion è circa il **25%** dell'errore commesso da SZZ.

7.8.5 Discussione

- A differenza di quanto visto nelle altre RQ, esiste poca variazione interna tra le varie implementazioni di SZZ.
- La qualità del labeling influenza direttamente la capacità di identificare le metriche software davvero rilevanti.

Solo nelle slide (non spiegato a voce)

Anche il metodo basilare **Simple** risulta più accurato rispetto a tutti i metodi SZZ in questo task specifico, arrivando a dimezzarne gli errori.

7.9 Figure, tabelle e workflow del paper

Questo capitolo descrive formalmente i workflow visuali presentati nelle slide e nel paper.

7.9.1 Diagramma del defect lifecycle

La figura sul **defect lifecycle** mostra la linea temporale che va da **IV** a **FV**, evidenziando il momento della **Ticket Creation** corrispondente a **OV** e il **Fix Commit** successivo.

7.9.2 Workflow del predicted AV

Il workflow relativo al **predicted AV** descrive il processo seguito in RQ2: i **Bug ID** collegano Jira, da cui si ricava la creazione del ticket, e Git, da cui si ricava il **Fix Hash**. A partire da queste informazioni, il metodo predice una AV che viene poi confrontata con l'**Actual AV**, usata come ground truth.

7.9.3 Workflow del class labeling

Il workflow del **class labeling**, utilizzato in RQ3, associa i file Java o Python toccati dai bug alle rispettive release per verificare i casi di **TP**, **TN**, **FP** e **FN**.

7.9.4 Workflow dei complete datasets

Il processo di creazione dei **complete datasets** estrae dal codice una serie di metriche, tra cui:

- **Size (LOC)**;
- **LOC Touched**;
- **Nfix**;
- **Churn**;
- **Age**;
- **Change Set Size**.

7.9.5 Feature selection e correlation analysis

I workflow successivi mostrano:

- il processo di **feature selection** tramite *Exhaustive Search*;
- l'applicazione della **correlazione di Spearman** tra la feature, ad esempio LOC, e la probabilità di difetto;
- la produzione finale del grafico del **Mean Relative Error**.

7.9.6 Interpretazione di grafici, boxplot e tabelle

I **boxplot** e le **tabelle comparative** confermano visivamente la superiorità strutturale dei metodi **Proportion**, in particolare **Proportion_MovingWindow** e **Proportion_Increment**, rispetto a SZZ in metriche come **MCC** e **Kappa**. Inoltre, la tabella sulla deviazione standard mostra che la deviazione standard di **P** è nettamente inferiore rispetto a quella di **IV**, **OV** e **FV**, rafforzando l'ipotesi di stabilità del ciclo di vita dei difetti.

7.10 Conclusioni e future work

7.10.1 Risultati principali del paper

I risultati principali sono:

1. l'**Affected Version** non può essere usata nel **51%** dei difetti;
2. il metodo **Proportion** è mediamente più accurato di SZZ per etichettare le classi difettose.

7.10.2 Perché AV non basta

Molto spesso gli sviluppatori non compilano il campo AV, oppure lo compilano in maniera inconsistente rispetto alle date di rilascio.

7.10.3 Perché Proportion è rilevante

L'ipotesi che i bug abbiano un **life-cycle stabile** risulta supportata dai dati: la deviazione standard di P all'interno dello stesso progetto è molto bassa. Proportion non solo supera SZZ nel labeling, ma commette anche una frazione molto più piccola dell'errore nell'identificare le feature ingegneristiche rilevanti.

7.10.4 Significatività statistica vs rilevanza pratica

Combinare le informazioni sul ciclo di vita dei defect con i metodi SZZ tradizionali produce un miglioramento **statisticamente significativo**, ma **praticamente trascurabile**, inferiore all'1%.

7.10.5 Sviluppi futuri

Le linee future di ricerca includono:

- analizzare commit bug-introducing successivi in SZZ;
- studiare l'accuratezza delle Affected Versions etichettate dagli sviluppatori Jira;
- replicare l'esperimento nel contesto della **Just-In-Time defect prediction**;
- testare combinazioni più raffinate tra metodi **Proportion** e **SZZ**.

In sintesi

Lo studio dimostra che i dataset di difettosità costruiti esclusivamente con `git blame`, quindi con SZZ, risultano inaccurati. Sfruttare invece una **costante di proporzionalità temporale** del ciclo di vita dei difetti, come avviene nei metodi **Proportion**, permette di costruire dataset più precisi e di migliorare sensibilmente la capacità dei modelli di Machine Learning di individuare i veri predittori dei bug nel codice.